Computers & Security 169 (2026) 104987

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

PacketPatch: Practical generation and deployment of adversarial packets for byte-feature-based encrypted traffic classification

Yuwei Xu [a,b,*], Yuanyuan Xu [a], Yunpeng Bai [a], Jiahui Chen [a], Kehui Song [c], Jie Cao [d], Qiao Xiang [e], Guang Cheng [a,b]

[a] School of Cyber Science and Engineering, Southeast University, Nanjing, 211189, China
[b] Purple Mountain Laboratories, Nanjing, 211111, China
[c] School of Software, Tiangong University, Tianjin, 300387, China
[d] Department of Electrical and Computer Engineering, Queen's University, Kingston, K7L 3N6, Canada
[e] School of Informatics, Xiamen University, Xiamen, 361005, China

ARTICLE INFO

Keywords:
Encrypted traffic classification
User privacy protection
Adversarial examples
Black-box assumption
Byte features
BERT

ABSTRACT

In recent years, byte features have been widely adopted in deep learning-based encrypted traffic classification, achieving remarkable performance. While enabling efficient network resource scheduling and enhancing network security, such techniques also pose serious threats to user privacy. To mitigate these risks, generating adversarial packets that evade traffic classifiers has emerged as an effective defensive strategy. However, existing studies face challenges including dependence on unrealistic preconditions, excessive time and bandwidth overhead, and compromised packet usability. To address shortcomings in prior studies, we present **PacketPatch**, a practical scheme for generating adversarial packets against byte-feature-based encrypted traffic classification (B-ETC). To avoid unrealistic preconditions, we design a BERT-based perturbation vector generator, PatchGenerator, that operates under a black-box assumption by producing perturbations from original packets and random header data without requiring access to model gradients or user behavior history. To reduce time and bandwidth overhead, PatchGenerator generates perturbations with a single forward pass and dynamically adjusts the number of masked bytes according to the packet size, thereby controlling the perturbation length. To ensure packet usability, we further propose a symmetric proxy-based deployment method. By inserting the perturbation vector at the boundary between the packet header and payload and updating the necessary protocol fields, adversarial packets can successfully pass integrity checks and be seamlessly restored at the receiving end. We evaluate PacketPatch on three datasets and compare it with four state-of-the-art white-box baseline methods. The results show that PacketPatch outperforms the baseline methods in most scenarios. Its average adversarial vector generation time is only 10.078 ms, and its bandwidth overhead is significantly lower than that of template-filling methods. Moreover, experiments on a prototype system confirm that PacketPatch can bypass B-ETC models and restore the original information at the receiver.

1. Introduction

The Internet has permeated numerous facets of contemporary society, playing a pivotal role in essential sectors such as finance, healthcare, and education (Aouedi et al., 2025). Consequently, an immense amount of sensitive personal information is continually transmitted across network infrastructure. As the fundamental conduit for information exchange, network traffic is critical to upholding data security and safeguarding privacy (Sharma and Lashkari, 2025). To bolster privacy protection in network communications, a range of encryption protocols, including HTTPS and QUIC, have been widely adopted (Dong et al., 2025).

The rapid evolution of artificial intelligence technology has led to the emergence of deep learning-based encrypted traffic classification (ETC) methods (Sharma and Lashkari, 2025), which are advancing at an impressive rate. These techniques have demonstrated remarkable precision in distinguishing various types of encrypted traffic across a range of network application scenarios, including web browsing (Cheng et al., 2021), mobile application access (Aceto et al., 2019), and Internet of Things transmission (Zhu et al., 2023). Although communication content is protected by encryption protocols, ETC methods can still

* Corresponding author at: School of Cyber Science and Engineering, Southeast University, Nanjing, 211189, China.
E-mail addresses: xuyw@seu.edu.cn (Y. Xu), xuyuanyuan@seu.edu.cn (Y. Xu), baiyp@seu.edu.cn (Y. Bai), jiahuichen@seu.edu.cn (J. Chen), songkehui@tiangong.edu.cn (K. Song), jie.cao@queensu.ca (J. Cao), qiaoxiang@xmu.edu.cn (Q. Xiang), chengguang@seu.edu.cn (G. Cheng).

https://doi.org/10.1016/j.cose.2026.104987
Received 3 February 2026; Received in revised form 5 May 2026; Accepted 21 May 2026
Available online 26 May 2026
0167-4048/© 2026 Published by Elsevier Ltd.

infer user behavior by leveraging the multi-dimensional features of traffic, posing a potential threat to personal privacy. In actual network environments, skilled adversaries can leverage advanced ETC models to infer sensitive information about access behaviors from encrypted traffic, thereby compromising user privacy (Papadogiannaki and Ioannidis, 2021).

In the realm of encrypted traffic classification, byte feature-based methods have emerged as the leading technical approach (Lin et al., 2022). These methods use natural language processing (NLP) concepts to treat structured packet byte sequences like text, allowing them to capture both local byte patterns and long-range contextual information. By examining packet headers and some payloads, these methods extract detailed semantic features from encrypted traffic, leading to better classification performance in various scenarios (Wu et al., 2023). Owing to their remarkable advantages in accuracy and applicability, byte-feature-based encrypted traffic classification (B-ETC) methods have progressively emerged as a significant tool for malicious attackers to infer user behavior and compromise sensitive information, thereby presenting fresh challenges to current encryption-based network privacy protection approaches (Shen et al., 2022).

In recent years, deep learning models have demonstrated an inherent vulnerability to adversarial examples (AEs) across domains like computer vision (CV) (Dai et al., 2024) and NLP (Ye et al., 2023). Even minor, thoughtfully crafted alterations can dramatically skew the model's decision-making process. Drawing inspiration from this, the development of adversarial examples aimed at undermining the inference abilities of B-ETC models presents a viable strategy for reducing the threat to user privacy. While recent research (Ding et al., 2022; Sadeghzadeh et al., 2021; Ding et al., 2024; Zhang et al., 2020; Hore et al., 2025; Sun et al., 2025; Kotak and Elovici, 2023; Cao et al., 2024) has delved into this area, the majority concentrates on improving AE success rates against classification models, usually ignoring practical limitations in real-world networks. The specific shortcomings of existing methods can be summarized in the following three aspects.

(1) **Unrealistic preconditions.** Current methods for generating adversarial examples frequently rely on idealistic assumptions. Some methods assume the privacy-preserving party has full access to the target ETC model, including its structure, parameters, and gradient information, allowing for constructing adversarial perturbations in a white-box manner (Ding et al., 2024; Sun et al., 2025; Kotak and Elovici, 2023). Some methods utilize a user's past network behavior, including previously accessed websites or applications, to steer the optimization of adversarial perturbations (Sadeghzadeh et al., 2021). In real-world network environments, a privacy-preserving party is unlikely to obtain such complete information.

(2) **Excessive overhead.** Although existing studies reduce the accuracy of B-ETC models, they often introduce significant time and bandwidth overhead. Some methods use several rounds of queries to the target model, adjusting perturbations based on feedback until they successfully mislead the model (Zhang et al., 2020; Hore et al., 2025; Sun et al., 2025). This interactive generation process is resource-intensive and suffers from substantial response latency, posing challenges in fulfilling the requirements for real-time transmission. Some methods improve the perturbation effect by adding many redundant bytes to packets, but this greatly increases traffic volume and wastes bandwidth (Ding et al., 2024; Sun et al., 2025; Kotak and Elovici, 2023).

(3) **Disrupting packet usability.** Unlike images, network data packets are strictly constrained by protocols at both the format and semantic levels. In network communication scenarios, even minor modifications to data packet fields can disrupt their normal functionality. Changes to packet header fields can disrupt routing, connection maintenance, and transmission control, while altering the payload can hinder the receiving end's ability to read application-layer data. Nevertheless, existing methods inadequately account for these protocol constraints, leading to adversarial examples that can deceive ETC models but do not facilitate normal data transmission (Ding et al., 2022).

Aiming at the shortcomings of previous work, we present **PacketPatch**. This innovative scheme effectively generates and deploys adversarial packets[1] in real network environments, thereby safeguarding user privacy from B-ETC models. In PacketPatch, we design a packet-level perturbation vector generator and propose a symmetric proxy-based deployment method. The main contributions of this paper can be summarized as the following points.

- To bypass unrealistic preconditions, we design PatchGenerator, a perturbation vector generator under the strict black-box assumption. PatchGenerator is built upon BERT and is trained using the same category prediction (SCP) task as well as the span-masked byte modeling (Span-MBM) task. It can capture deep semantic information from packet bytes, removing reliance on the target model's gradient information. Moreover, by utilizing the original packet alongside a random header as input and defining the optimization goal to increase the disparity from the original packet while aligning the generated features closer to the random header, the generator eliminates the necessity to obtain prior insights into user access behavior.
- To reduce time and bandwidth overhead, we focus on comprehensive optimization of PatchGenerator. By generating perturbation vectors through a single forward pass, PatchGenerator avoids the delays associated with multiple iterative queries, thus ensuring real-time performance. By adjusting the number of mask bytes based on packet size, PatchGenerator manages the perturbation vector length effectively. Thus, it safeguards against high bandwidth overheads that may be incurred from unnecessarily padding with excessive bytes.
- To ensure the usability of adversarial packets, a symmetric proxy-based deployment method is proposed. At the sending end, the proxy inserts a perturbation vector at the junction of the header and payload, effectively safeguarding against message transmission failures that may arise from corruption in the header fields. Both the packet length and checksum fields are adjusted simultaneously to ensure that adversarial packets can pass the integrity check upon reaching their destination. At the receiver's end, the proxy precisely eliminates the perturbation vector based on the synchronized configuration, guaranteeing the accurate interpretation of the original information.
- To validate PacketPatch, we executed comprehensive comparative experiments across three datasets, comparing its effectiveness with four adversarial packet generation methods. The results show that PacketPatch outperforms current state-of-the-art white-box baselines in most scenarios. Moreover, PacketPatch produces perturbation vectors in just 10.078 ms, well within acceptable network transmission latency limits, while achieving a bandwidth overhead that is considerably lower than that of template-filling methods. Finally, tests on a prototype system show that PacketPatch's adversarial packets can successfully bypass B-ETC models and allow the receiving end to accurately recover the original information.

[1] In the general machine learning literature, adversarial examples refer to inputs intentionally perturbed to mislead models. In this work, we consider a specific instantiation in the network traffic domain, namely adversarial packets, which must additionally satisfy protocol compliance, transmission validity, and recoverability constraints.

The rest of this paper is organized as follows. In Section 2, we summarize previous related research, covering B-ETC models and their adversarial example generation methods. The overall design of PacketPatch is introduced in Section 3, following the presentation of the threat model and design goals. In Section 4, we describe PatchGenerator, focusing on its model architecture, training tasks, and the generation of perturbation vectors. Section 5 delves into the practical implementation of adversarial packets, covering system deployment, the construction of adversarial packets, and the restoration of original information. We detail the experimental setup in Section 6 and evaluate PacketPatch's performance, overhead, and usability in Section 7. Finally, a conclusion is given in Section 8.

2. Related work

This section begins with an exploration of the technological advancements in B-ETC methods, followed by a comprehensive overview of the existing adversarial example generation techniques for these classification models.

2.1. B-ETC methods

Recently, byte features of packets have gained increasing attention in ETC tasks due to their ability to bypass complex feature extraction processes while achieving high classification accuracy. Depending on the data representation strategy, existing B-ETC methods can be categorized into three classes: image-based, sequence-based, and contextualized representations.

Some researchers convert the content of packets into image formats and apply deep learning models originally developed for CV to complete traffic classification. At the packet level, the authors of Zhou et al. (2021) mapped the first 1024 bytes of a packet into a 32×32 grayscale image, employing 2D-CNNs for classification. Extending this to the session level, in Wang et al. (2017b), the authors pioneered the conversion of the first 784 bytes of a session into 28×28 grayscale images, demonstrating that treating raw traffic as visual inputs effectively facilitates malware detection. To further enrich feature expressiveness, the authors of Hu et al. (2023) transformed session bytes into 28×28 RGB images. By utilizing multi-channel color encoding rather than simple grayscale intensity, this method captures more complex, fine-grained patterns essential for identifying malware variants.

Acknowledging that network traffic intrinsically consists of sequential data streams, some methods focus on sequence-based representation. At the packet level, in Wang et al. (2017a), the authors provided empirical evidence that 1D-CNNs exhibit feature extraction capabilities superior to 2D-CNNs when applied to raw traffic bytes. Capitalizing on this finding, DeepPacket (Lotfollahi et al., 2020), an ETC model utilizing 1D-CNN architecture, was proposed. It processes the entire 1500-byte maximum transmission unit (MTU) of each packet as a unified sequence for classification. For flow-level analysis, TSCRNN (Lin et al., 2021) serves as a hybrid framework that synergizes 1D-CNNs with Bi-LSTMs. By processing the first 15 packets of a flow, this model concurrently extracts intra-packet spatial features and inter-packet temporal dependencies, yielding robust spatiotemporal representations.

Emerging studies have adopted contextualized representation, inspired by NLP, to capture deep semantic dependencies by regarding traffic bytes as semantic tokens. PERT (He et al., 2020) utilizes the ALBERT architecture combined with bigram tokenization to extract dynamic contextual features from packet payloads. Leading this direction, the authors of Lin et al. (2022) proposed ET-BERT, which adapts the pre-training paradigm to massive unlabeled datasets. By encoding packet bytes into discrete tokens, this method learns contextualized representations of network protocols, effectively modeling the bidirectional context within the traffic.

2.2. AE against B-ETC methods

Due to the discrete, non-differentiable nature of network traffic and rigid protocol syntax, constructing adversarial examples for packet bytes is challenging. Existing methods are categorized into gradient-based, query-based, and saliency-based methods, as summarized in Table 1.

To leverage the differentiability of target models, gradient-based methods typically employ optimization strategies that relax the discrete constraints of network traffic by mapping packet bytes into continuous image pixel matrices to compute perturbation directions. Targeting general scenarios, P2I-Adv (Ding et al., 2022) processes packet content into 2D grayscale images and employs mature CV algorithms like FGSM to generate perturbations. However, this method performs irreversible modifications on packet headers. Since network packets have strict structural constraints, such modifications disrupt packet usability and often prevent the receiver from parsing the information. To address the universality issue, the authors of Sadeghzadeh et al. (2021) proposed ANT, which trains specific perturbation vectors for each traffic class and adds them to the packet tail. Nevertheless, its drawback lies in the unrealistic precondition that it necessitates prior knowledge of the user's browsing behavior, specifically the traffic category, to select targeted parameters. Similarly, UTAA (Ding et al., 2024) generates fixed-length perturbations of 784 bytes based on gradients and applies them via modulo addition. This fixed-length strategy forces the padding of excessive bytes when perturbing short packets, leading to unacceptable bandwidth consumption. In summary, although these methods are computationally efficient, they fundamentally rely on white-box access to the attacker's model, including model structure and gradients. This requirement limits their application in realistic defense scenarios where model parameters are usually unknown.

In scenarios where gradient information is inaccessible, query-based methods serve as a viable alternative, relying on iterative processes that employ heuristic strategies or reinforcement learning (RL) to explore the perturbation space by continuously probing the target model. In Zhang et al. (2020), the authors analyzed browser parsing mechanisms and proposed MCTS-T, utilizing Monte Carlo Tree Search to iteratively construct adversarial examples that bypass XSS detectors. Expanding on this idea for general traffic, the authors of Hore et al. (2025) designed DeepPackGen, an RL-based generator that modifies specific header fields to bypass detection. Another RL-based method, AdvTg (Sun et al., 2025), tends to introduce excessive padding to ensure evasion, which results in prohibitive costs regarding bandwidth efficiency. The common limitation of this category is the excessive overhead in time consumption. These methods rely on iterative queries to the classification model, refining perturbations based on feedback until deception is achieved. This process is extremely time-consuming, rendering such methods unsuitable for real-time transmission. Furthermore, the reliance on continuous interaction implies they are not strictly black-box methods, as they require a responsive query interface to the specific target model.

Distinct from global perturbation strategies, saliency-based methods focus on feature replacement by prioritizing the manipulation of specific critical tokens identified by interpretability tools or vulnerability analysis. A-IOT (Kotak and Elovici, 2023) processes the first 784 bytes into a grayscale image and uses Class Activation Mapping (CAM) to rank pixel importance, retaining only critical bytes in a transmission template. A major downside of this approach is its use of a fixed-length template of 784 bytes, which causes excessive bandwidth consumption when processing shorter packets. More recently, the authors of Cao et al. (2024) introduced BYTEFLIPPING, which utilizes a payload byte vulnerability ranking (PBVR) algorithm to identify and perturb vulnerable bytes specifically. Although some methods in this category attempt to optimize byte selection to preserve semantics, they share a common unrealistic precondition. Calculating saliency maps or vulnerability rankings inherently necessitates white-box access to the attacker's model, such as internal activation functions or detailed probability outputs, which is rarely available to defenders in practice.

Table 1
A summary of adversarial example methods against B-ETC models.

Ref.	Method	Category	Black-Box	No-User-Info	Real-Time	Limited BWO	Usability
Ding et al. (2022)	P2I-Adv	Gradient-based	×	✓	✓	✓	×
Sadeghzadeh et al. (2021)	ANT		×	×	✓	✓	✓
Ding et al. (2024)	UTAA		×	✓	✓	×	✓
Zhang et al. (2020)	MCTS-T	Query-based	×	✓	×	✓	✓
Hore et al. (2025)	DeepPackGen		×	✓	×	✓	✓
Sun et al. (2025)	AdvTg		×	✓	×	×	✓
Kotak and Elovici (2023)	A-IOT	Saliency-based	×	✓	✓	×	✓
Cao et al. (2024)	BYTEFLIPPING		×	✓	×	✓	✓
Ours	**PacketPatch**	**Inference-based**	✓	✓	✓	✓	✓

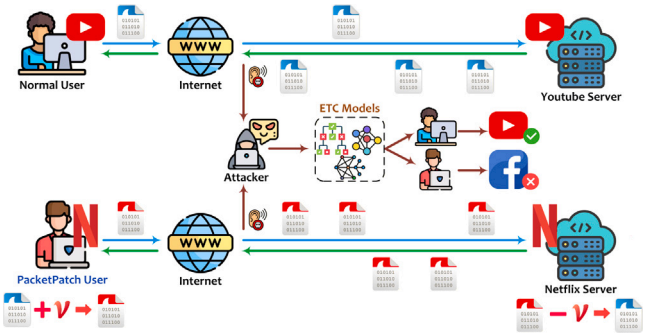


Fig. 1. The privacy protection scenario of PacketPatch.

3. Design of PacketPatch

To systematically address the practical limitations of existing adversarial attacks, this section articulates the comprehensive design of PacketPatch. We first formally define the threat model to establish a rigorous boundary for the defense, followed by the formulation of critical design goals. Finally, we present the overview of the PacketPatch framework, demonstrating how these goals are realized through a coherent operational workflow.

3.1. Threat model

We consider a typical network communication environment, as illustrated in Fig. 1. In this scenario, network packets are transmitted between client endpoints and remote servers via public networks to deliver internet services. However, this transmission path is inherently vulnerable. Adversaries located at intermediate nodes, such as routers, gateways, and switches, can capture the passing traffic. By leveraging well-trained ETC models, these adversaries analyze packet content to infer sensitive user activities, resulting in privacy leakage.

To formally characterize this threat, we define the problem as follows. Let $\mathcal{X}$ represent the space of network packets, where a sensitive packet is denoted as $x \in \mathcal{X}$. Let $\mathcal{Y} = \{1, 2, \ldots, K\}$ denote the set of traffic categories, where $y \in \mathcal{Y}$ represents the ground-truth label associated with x. The adversary deploys a well-trained B-ETC model, denoted as $C : \mathcal{X} \to \mathcal{Y}$, which maps the observed packet x to a predicted label. We define the adversary $\mathcal{A}$ as a passive eavesdropper characterized by the following three aspects:

Attacker's Goal: The primary objective of $\mathcal{A}$ is to infer the category of the user's encrypted traffic, thereby facilitating the construction of user profiles or website fingerprinting attacks. To achieve this, $\mathcal{A}$ employs C to predict the label of a captured packet x. The adversary aims to maximize the expected classification accuracy over the underlying data distribution D. As shown in Eq. (1), where $\mathbb{I}(\cdot)$ denotes the indicator function, this optimization problem is formulated as:

$$\max \mathbb{E}_{(x,y)\sim D}\left[\mathbb{I}(C(x) = y)\right]. \tag{1}$$

Attacker's Knowledge: We adopt a realistic threat setting where $\mathcal{A}$ possesses domain knowledge regarding the network environment. $\mathcal{A}$ is fully aware of the traffic categories and protocols present in the network and has trained a high-performance C by leveraging abundant publicly available labeled datasets. However, $\mathcal{A}$ is oblivious to the existence of any active defense mechanisms, operating under the assumption that the captured traffic consists purely of benign packets rather than adversarial packets.

Attacker's Capability: The adversary $\mathcal{A}$ maintains read-only access to the traffic channel. They can capture, store, and analyze packets flowing through the public network but cannot modify, drop, or inject packets.

Under this threat model, the defender aims to preserve user privacy by generating a perturbation vector v and applying it to the original packet x to construct an adversarial packet $\tilde{x}$. Consequently, the defender seeks to minimize the expected probability that the attacker successfully identifies the true label y across the data distribution. As formulated in Eq. (2), this optimization goal is defined as:

$$\min \mathbb{E}_{(x,y)\sim D}\left[\mathbb{I}(C(\tilde{x}) = y)\right]. \tag{2}$$

3.2. Design goals

To effectively counter the threats defined in Section 3.1, we delineate five critical design goals for PacketPatch:

(1) Achieve Black-Box Applicability: PacketPatch must function strictly without access to the target identification model's internal parameters, gradients, or training data. The defense mechanism relies solely on the input packet features to construct adversarial packets, ensuring robustness in realistic scenarios where attackers conceal their model architectures.

(2) Ensure User-Agnostic Operation: The perturbation generation must rely exclusively on the features of the current packet, independent of any user-specific private information or historical behavioral profiles. This constraint ensures that the privacy protection mechanism itself does not become a source of sensitive data leakage.

(3) Guarantee Real-Time Efficiency: To support high-throughput network environments, the adversarial packet generation process must achieve millisecond-level latency, strictly capping the processing time below 20 ms per packet. The inference overhead must be negligible to ensure that the defense mechanism does not introduce perceptible delays to the user's communication service.

(4) Maintain Controllable Bandwidth Overhead: The system must strictly limit the size of injected perturbation vectors to avoid network congestion. Specifically, the bandwidth overhead is constrained to a maximum ratio of 10% relative to the original packet size. Furthermore, the total packet length after perturbation is strictly bounded by the standard MTU of 1500 bytes to prevent IP fragmentation or packet drops.

(5) Ensure Packet Validity and Recoverability: The modified packets must strictly adhere to standard network protocol specifications to avoid being filtered by intermediate devices. Crucially, the scheme necessitates a reversible mechanism, ensuring that the legitimate receiver can perfectly remove the perturbation and reconstruct the original payload for correct content parsing.

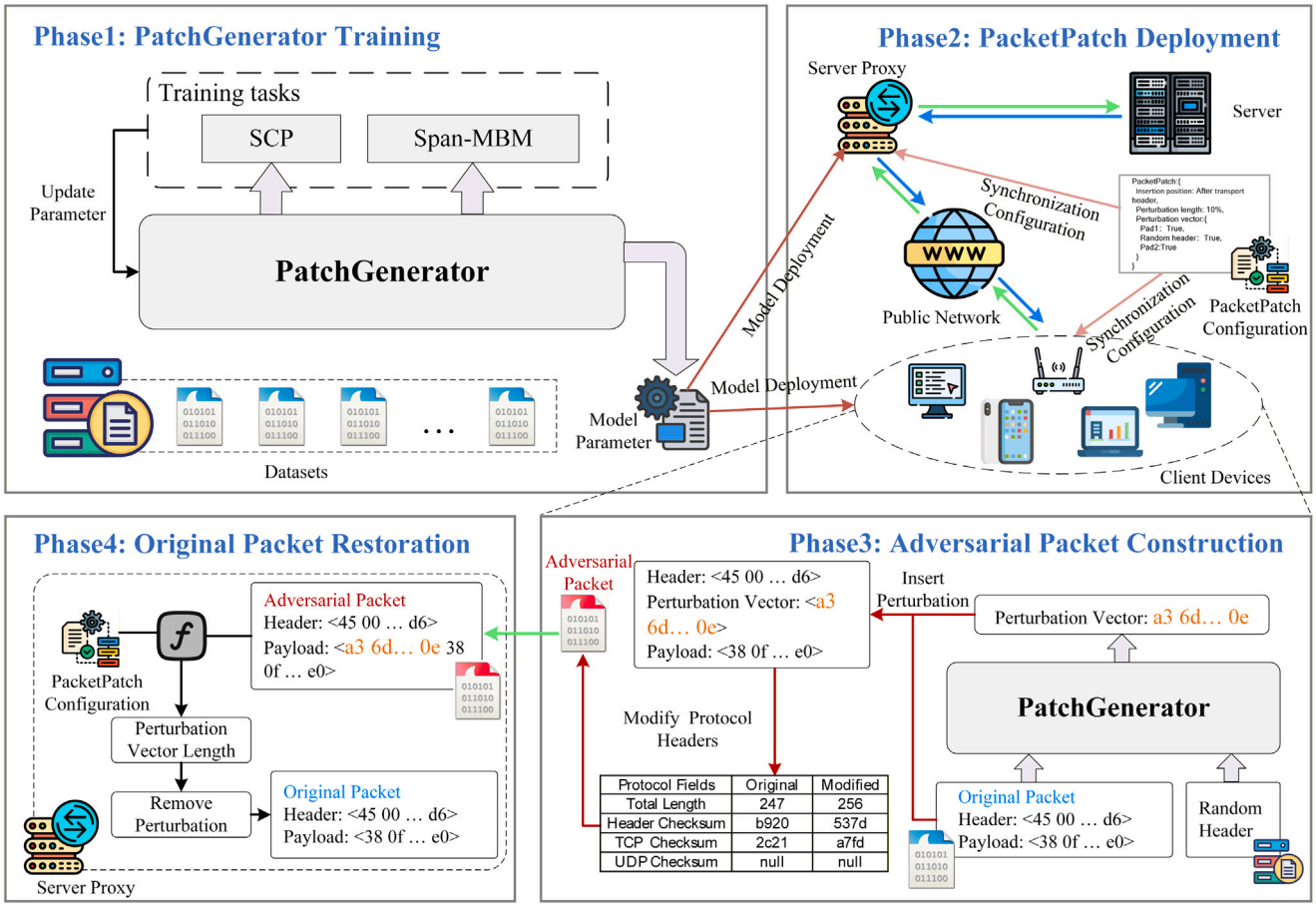


Fig. 2. The workflow of PacketPatch.

3.3. Overview of PacketPatch

To effectively achieve the design goals outlined in Section 3.2, we propose **PacketPatch**, a practical adversarial packet construction framework. Structurally, PacketPatch comprises a perturbation vector generator, named PatchGenerator, and a symmetric proxy-based deployment mechanism, which jointly facilitate privacy preservation against B-ETC models while maintaining strict packet recoverability. As illustrated in Fig. 2, the system workflow is systematically organized into four distinct phases:

(1) PatchGenerator Training. We design the PatchGenerator based on the ET-BERT architecture (Lin et al., 2022). To equip this architecture with the specific capability to generate adversarial perturbations, we propose two customized training tasks. The SCP task is designed to capture discriminative features for traffic classification. Meanwhile, the Span-MBM task is formulated to provide context-aware reconstruction capabilities, allowing the model to infer and generate byte sequences consistent with the packet context.

(2) PacketPatch Deployment. The well-trained PatchGenerator is independently deployed on both the client side and the server side. To ensure the reversibility of the perturbation, the configuration file of PacketPatch is synchronized between both parties, defining the critical parameters used for constructing adversarial packets. To ensure non-invasive integration with existing network applications, the deployment is performed through a proxy architecture. This design allows for seamless packet manipulation without requiring modifications to the original application logic.

(3) Adversarial Packet Construction. Due to the bidirectional nature of network traffic, both the user side and the server side alternately function as the sender and receiver. Before transmitting data, the sender utilizes the deployed PacketPatch system to construct an adversarial packet. The system generates and embeds the perturbation vector into the payload, and the resulting packet is then transmitted across the network.

(4) Original Packet Restoration. Upon receiving the adversarial packet, the receiver activates the restoration process according to the synchronized configuration file. The system removes the perturbation to restore the original packet perfectly. Subsequently, the restored data is parsed following standard protocols, ensuring that the original information is delivered correctly to the application layer.

4. PatchGenerator

This section presents the PatchGenerator. First, we introduce the architecture and input representation optimized for packet bytes. Then, we detail the SCP and Span-MBM tasks used to train the model. Finally, we describe the procedure for generating context-aware perturbation vectors.

4.1. Model architecture of PatchGenerator

Given the powerful encrypted traffic feature extraction capabilities of the pre-trained ET-BERT model, we adopt ET-BERT as the foundation for PatchGenerator. As illustrated in Fig. 3, the architecture of PatchGenerator is structurally organized into four primary components: the input layer, the embedding layer, the encoder layer, and the task-specific output layer.

(1) Input Layer. The input layer processes raw bytes from two distinct packets into discrete tokens. Following the ET-BERT design, every two consecutive bytes are treated as a single input unit and tokenized

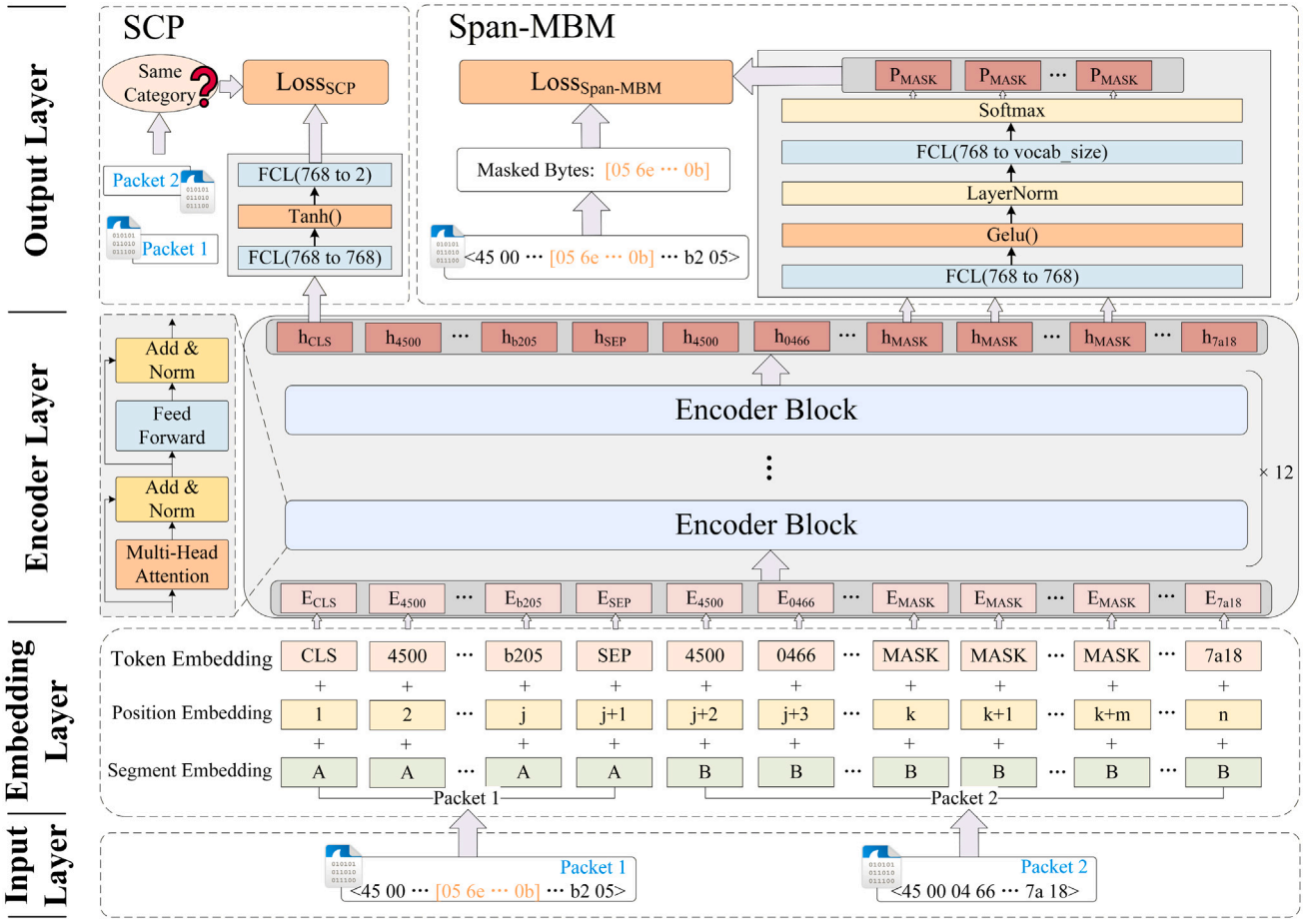


Fig. 3. Model architecture of PatchGenerator.

via byte-pair encoding (BPE). Diverging from the ET-BERT approach, we optimize the input representation by increasing the sliding window stride from 1 to 2. This adjustment creates non-overlapping input units, allowing the model to cover a broader context of the packet bytes.

(2) Embedding Layer. To capture multidimensional features of the tokenized inputs, three distinct embedding methods are employed. Token embedding maps the semantic information of byte tokens into dense vectors. Position embedding encodes the sequential order of tokens to preserve positional information within the input sequence. Segment embedding assigns segment A to *Packet 1* and segment B to *Packet 2*, enabling the model to distinguish which packet each token belongs to. All three embeddings share a dimension of 768 and are summed element-wise to form the input for the subsequent encoder layer.

(3) Encoder Layer. Serving as the core feature extractor, this layer employs 12 transformer encoder blocks. Within each block, the input first passes through a multi-head self-attention mechanism with 12 heads to capture global dependencies among byte tokens, then through a position-wise feed-forward network. To facilitate deep network training, residual connections and layer normalization are applied after each sub-layer.

(4) Output Layer. The output layer comprises two task-specific heads designed for distinct purposes. To capture discriminative features across traffic categories, the SCP head first extracts the [CLS] token representation $h_{\texttt{CLS}}$, then predicts categorical consistency between *Packet 1* and *Packet 2* through fully connected layers (FCLs) with Tanh activation. To enable context-aware byte reconstruction, the Span-MBM head first extracts the masked token states $h_{\texttt{MASK}}$, then generates probability distributions over the token vocabulary through an FCL with Gelu activation, layer normalization, and a softmax decoder.

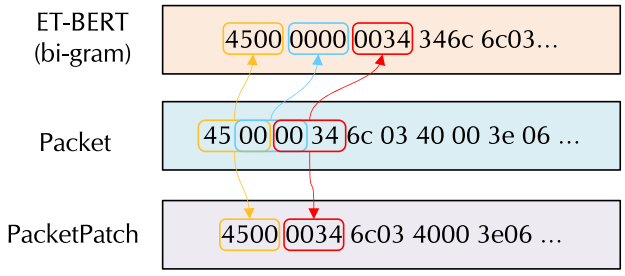


Fig. 4. Comparison of byte embedding mechanisms between ET-BERT and PatchGenerator.

4.2. Input representation of PatchGenerator

The standard ET-BERT employs bi-gram encoding with a sliding window stride of 1. While robust for classification, this representation imposes critical limitations on perturbation generation. The inherent token overlap causes significant content redundancy that complicates packet reconstruction, while the dense stride restricts the effective contextual coverage to only $n + 1$ bytes for a sequence of length n, hindering the capture of long-range dependencies. To address these inefficiencies, we introduce a strategic optimization to the input encoding, as illustrated in Fig. 4. While retaining the bi-gram unit structure, we increase the sliding window stride from 1 to 2. This adjustment creates a non-overlapping input sequence, fundamentally altering how the model interacts with raw packet bytes.

This modification yields three critical advantages. First, the elimination of overlap ensures that adjacent input units no longer share bytes, allowing the generated vectors to be directly interpreted as continuous byte sequences without the need for post-processing. Second, this approach provides extended coverage, where a model input of length n tokens can now process $2n$ bytes, effectively doubling the contextual information available for perturbation generation. Finally, the modified input representation preserves pre-trained knowledge by sharing half of its content with the original ET-BERT format. This structural alignment ensures that the pre-trained features remain transferable, while the subsequent training process further adapts the PatchGenerator to this optimized input form.

4.3. SCP task

To enable PatchGenerator to capture discriminative features across traffic categories, we design the SCP task, which trains the model to determine whether two input packets belong to the same traffic category. The core motivation is that effective adversarial perturbation generation requires the model first to understand categorical differences, as this capability is a prerequisite for generating perturbations that can successfully alter classification outcomes. The implementation of the SCP task proceeds in two sequential stages comprising input construction and model training.

Algorithm 1 The input construction of the SCP task.

Require: Collection of packets S, Maximum input length n
Ensure: Paired model input X

$X \leftarrow []$
for $(x_i, y_i) \in S$ **do**
  // Step 1: Packet Pairing
  $P_{sameCategory} \leftarrow random(0, 1)$
  if $P_{sameCategory} < 0.5$ **then**
    $(x_j, y_j) \leftarrow random(\{y_j \neq y_i, (x_j, y_j) \in S\})$
  else
    $(x_j, y_j) \leftarrow random(\{y_j = y_i, (x_j, y_j) \in S\})$
  end if
  // Step 2: Dynamic Partition
  $P_{x_i Start} \leftarrow random(0, 1)$
  if $P_{x_i Start} < 0.5$ **then**
    $p_{x_i end} \leftarrow random(0.25, 0.75)$
    $x_i \leftarrow x_i[0 : p_{x_i end} \times n]$
  else
    $p_{x_i start} \leftarrow random(0, 0.5)$
    $p_{x_i end} \leftarrow random(x_i start, 0.75)$
    $x_i \leftarrow x_i[p_{x_i start} \times n : p_{x_i end} \times n]$
  end if
  Update x_j similar to lines 9-17
  // Step 3: Sequence Assembly
  $x \leftarrow [].expand([CLS], x_i, [SEP], x_j)$
  if $len(x) > n$ **then**
    $x \leftarrow x[0 : n]$
  end if
  while $len(x) < n$ **do**
    $x.append([PAD])$
  end while
  $X.append(x)$
end for
  return X

(1) Input Construction. The input construction process is detailed in Algorithm 1. To enhance the model's generalization capability, we implement a three-step construction strategy:

- **Step 1: Packet Pairing:** For each packet (x_i, y_i) in the dataset, a second packet (x_j, y_j) is randomly selected. To ensure class balance, there is a 50% probability that x_j belongs to the same category as x_i and a 50% probability that it belongs to a different category.
- **Step 2: Dynamic Partition:** To prevent overfitting to specific header fields, each packet undergoes dynamic partition. With 50% probability, the packet is truncated from the header; otherwise, it starts from a random early position. The length is randomly selected between 1/4 and 3/4 of the maximum input length (n).
- **Step 3: Sequence Assembly:** The two packets are concatenated into a single sequence: `[CLS]` + x_i + `[SEP]` + x_j. The sequence is then padded with `[PAD]` tokens or truncated to strictly match the fixed input length n.

(2) Model Training. After obtaining the paired input set X, the model is trained to distinguish whether the packet pair shares the same category. For supervision, we assign the ground-truth label $y = 1$ to same-category pairs and $y = 0$ to different-category pairs. The model utilizes the hidden state at the `[CLS]` token position to generate the predicted probability $P(\text{same})$ that the two packets belong to the same category, and conversely $P(\text{diff})$ that they originate from different categories. Based on these definitions and the ground-truth label y, the training objective is to minimize the binary cross-entropy loss function, as defined in Eq. (3):

$$Loss_{SCP} = -[y \log P(\text{same}) + (1 - y) \log P(\text{diff})] \tag{3}$$

4.4. Span-MBM task

Standard Masked Language Modeling (MLM) (Devlin et al., 2019) relies on discrete masking, which is ill-suited for generating continuous adversarial perturbations. To bridge this gap, we design the Span-MBM task. By extending the masking mechanism from isolated tokens to continuous segments, this task forces the model to utilize long-range context, thereby equipping PatchGenerator with the capability to synthesize coherent, continuous byte sequences. The implementation proceeds in two stages: input construction and model training.

(1) Input Construction. The detailed construction process of long-range masks is presented in Algorithm 2. Specifically, for an input sample $x = \{t_1, t_2, \ldots, t_n\}$ with n tokens, we first determine the total number of tokens to be masked, denoted as N_{mask}. The process proceeds in two sequential steps corresponding to the algorithm:

- **Step 1: Span Length and Position Generation.** We iteratively generate the coordinates for each mask span until the cumulative length of masked tokens reaches the target count N_{mask}. In each iteration, a span length is first randomly sampled, followed by the selection of a starting position uniformly distributed across the sequence. To ensure the validity of the mask, we enforce a strict non-overlapping constraint: any candidate span that intersects with previously selected positions is discarded or re-adjusted.
- **Step 2: Span-Masked Byte Replacement.** After determining the positions, we execute the replacement. Crucially, to compel the model to recover continuous semantics, we apply a probabilistic strategy to the entire span as a whole unit, rather than to individual tokens. For each span, there is an 80% probability of replacing all internal tokens with `[MASK]`, a 10% probability of replacing them with random noise tokens, and a 10% probability of keeping the original tokens unchanged.

(2) Model Training. After obtaining the masked sample $\overline{x}$, the PatchGenerator is trained to reconstruct the original byte sequence based on the bidirectional context. We utilize an FCL to map the encoder output to the packet vocabulary space. To formulate the optimization objective, let $maskPos$ represent the set of all token positions within the selected spans, and $P(\cdot)$ denote the probability of correctly

Algorithm 2 The input construction of the Span-MBM task.

Require: Original sample $x = \{t_1, t_2, \ldots t_n\}$, Total number of mask N_{mask}
Ensure: Sample after replacing Span-Masked Bytes $\overline{x}$

// Step 1: Span Length and Position Generation
$cnt \leftarrow 0$
$spanPos \leftarrow \{\}$
while $cnt < N_{mask}$ **do**
  $spanLen \leftarrow random(0, N_{mask})$
  if $spanLen + cnt > N_{mask}$ **then**
    $spanLen \leftarrow N_{mask} - cnt$
  end if
  do
    $posStart \leftarrow random(0, n)$
    if $posStart + spanLen > n$ **then**
      $spanLen \leftarrow n - posStart$
    end if
    $span \leftarrow \{posStart, \ldots, posStart + spanLen\}$
  while $\{span \cap pos | \forall pos \in spanPos\} \neq \emptyset$
  $spanPos \leftarrow spanPos \cup \{span\}$
end while
// Step 2: Span-Masked Byte Replacement
$\overline{x} \leftarrow x$
for $pos \in spanPos$ **do**
  $p \leftarrow random(0, 1)$
  if $p < 0.8$ **then**
    for $i \in pos$ **do**
      $\overline{x}[i] \leftarrow [MASK]$
    end for
  else if $p < 0.9$ **then**
    for $i \in pos$ **do**
      $\overline{x}[i] \leftarrow getRandomCharacter()$
    end for
  else
    $continue$
  end if
end for
return $\overline{x}$

predicting the original token t_i. The training goal is to minimize the Negative Log-Likelihood loss, as defined in Eq. (4):

$$Loss_{Span-MBM} = -\sum_{i \in maskPos} \log(P(\overline{x}[i] = t_i | \overline{x}; \theta)) \tag{4}$$

To integrate the discriminative and generative capabilities developed by the two self-supervised tasks, we combine their respective optimization goals. The overall loss function guiding the PatchGenerator is calculated as the sum of the SCP loss and the Span-MBM loss, as shown in Eq. (5):

$$Loss = Loss_{SCP} + Loss_{Span-MBM} \tag{5}$$

The equal weighting of the two loss terms follows standard practice in BERT pre-training, where multiple objectives are combined without additional coefficients. Both losses are formulated as cross-entropy losses on the same numerical scale, which provides a balanced optimization signal without introducing extra hyperparameters.

The Span-MBM objective trains the model to learn the conditional distribution of packet bytes under the given context. This allows the model to generate tokens that follow the statistical patterns observed in the data. By selecting tokens from different parts of the learned distribution, the model can modify the statistical patterns of the packet. These statistical patterns correspond to the features used by B-ETC models for classification. Changing these patterns directly affects the model input, which increases the probability of misclassification. The SCP objective complements this process by enabling the model to capture differences between traffic categories, providing guidance on how these patterns vary across classes. Together, these two objectives provide both the ability to model byte-level patterns and the ability to control them, which supports the generation of effective adversarial perturbations. This enables the learned distribution to be directly utilized for perturbation generation.

4.5. Perturbation vector generation

Upon completing the training of the SCP and Span-MBM tasks, the PatchGenerator acquires dual capabilities: distinguishing traffic categories and predicting masked content based on context. This section details how to leverage these capabilities to generate adversarial perturbation vectors. As illustrated in Fig. 5, the generation process comprises the following four stages:

(1) Input Preparation. Given a target packet, we first separate its header x_h from its payload x_p. The header includes both network-layer and transport-layer headers. The perturbation length L_{pad} is set to 10% of the original packet length. This ratio is selected to strictly limit the bandwidth overhead introduced by the perturbation vector relative to the original packet size. For a standard packet with an MTU of 1500 bytes, this yields a perturbation budget of 150 bytes, corresponding to 75 tokens, which keeps the masked portion under 30% of the total input and preserves sufficient context for effective generation. The 10% setting consequently balances perturbation effectiveness against both bandwidth cost and generation quality. A sequence of L_{pad} `[MASK]` tokens is then prepared for insertion.

(2) Sequence Construction. A random index $r \in [0, L_{\text{pad}}]$ is selected to partition the `[MASK]` sequence into two segments: Pad_1 and Pad_2. A `[SEP]` token is inserted at this position as a segment delimiter, followed by a randomly sampled header h_r with a constant length of 40 bytes. The final input sequence S_{in} is constructed as:

$$S_{in} = [\texttt{CLS}] + x_h + Pad_1 + [\texttt{SEP}] + h_r + Pad_2 + x_p \tag{6}$$

This structure allows the PatchGenerator to treat the input as two logically segmented packet entities, leveraging contextual information from both the original and random packets. In practice, h_r is drawn uniformly from a pre-constructed header pool that aggregates standard 20-byte IP headers and 20-byte TCP headers extracted from the training dataset. Because the pool encompasses headers from all traffic categories, the probability that h_r shares the same category as the target packet is negligibly small. Consequently, no category label is required at inference time, and the selection is performed purely at random without relying on any user behavior information.

(3) Perturbation Generation. The sequence S_{in} is fed into the PatchGenerator, which outputs conditional probability distributions over candidate bytes at each masked position. These distributions reflect the statistical patterns of packet bytes under the given context. To generate adversarial perturbations, we apply distinct selection strategies to the two segments based on different parts of the learned distribution:

- **Pad_1:** The $\arg\min$ operation selects tokens with the lowest conditional probability under the context of the original header x_h. This introduces byte patterns that are inconsistent with the original traffic, thereby disrupting feature consistency.
- **Pad_2:** The $\arg\max$ operation selects tokens with the highest conditional probability under the context of the injected random header h_r. This generates byte patterns that are consistent with other traffic categories.

PatchGenerator models the conditional distribution of packet bytes given contextual information. The $\arg\min$ operation selects tokens from low-probability regions of this distribution, while the $\arg\max$ operation selects tokens from high-probability regions. Their combination

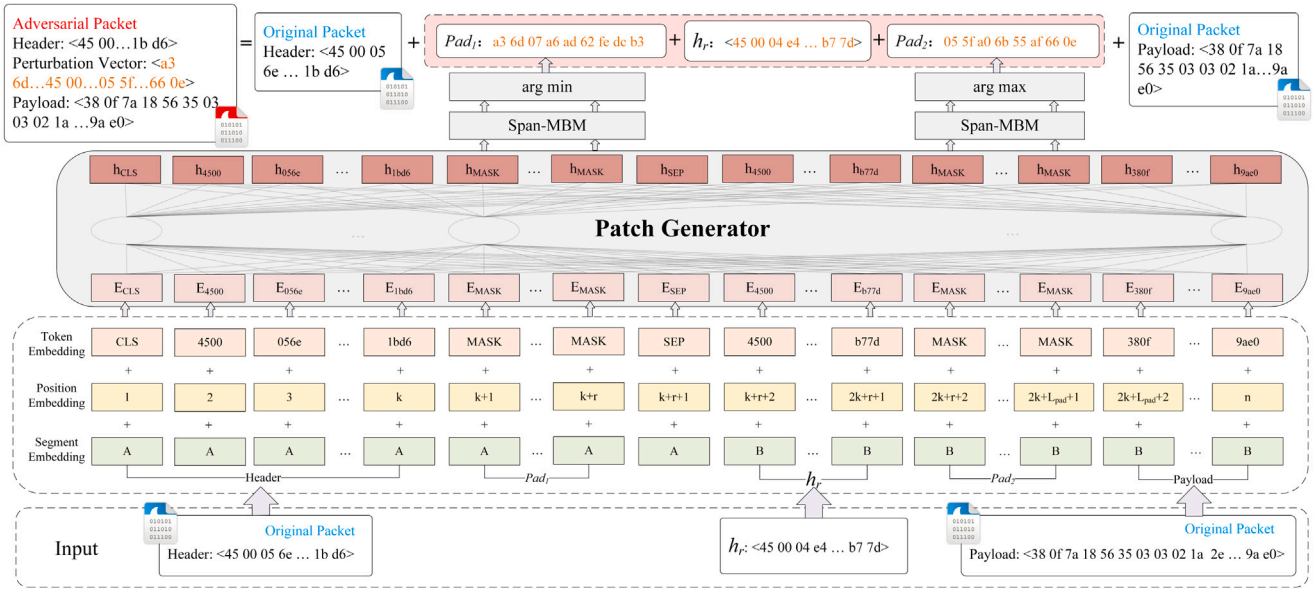


Fig. 5. The perturbation vector generation process of PatchGenerator.

changes the statistical patterns of the packet, pushing it away from the original class and toward other classes. Since B-ETC models rely on these statistical patterns to define decision boundaries, such changes increase the probability that the perturbed packet crosses the decision boundary. This process does not depend on any specific model structure and therefore remains effective in the black-box setting.

(4) Vector Assembly. The final perturbation vector v is assembled by concatenating three components:

$$v = Pad_1 + h_r + Pad_2 \tag{7}$$

The random header h_r is retained as it introduces realistic feature patterns from an alternative category. This combined vector is embedded between the transport-layer header and payload of the original packet, yielding the final adversarial packet. The total perturbation length is fixed at $L_{pad} + 40$ bytes, ensuring a unique mapping between the original and adversarial packet lengths for accurate recovery by the receiver.

5. Application of adversarial packets

This section details the practical implementation of the adversarial traffic application system. To validate the effectiveness of the proposed PacketPatch scheme in real-world network environments, we design and implement a prototype system based on a symmetric proxy architecture.

5.1. System architecture and deployment

Given the complexity of directly modifying network stacks within user applications, we adopt a non-intrusive symmetric proxy architecture based on the SOCKS5 protocol, as illustrated in Fig. 6. The system consists of two core components:

(1) Client Proxy: Deployed on the user's host, this component intercepts outbound traffic from client applications, such as web browsers. It constructs adversarial packets by embedding perturbations and forwards them to the server proxy.

(2) Server Proxy: Deployed on a remote server, this component receives the adversarial traffic, removes the perturbations to restore the original payload, and forwards the data to the target server.

To ensure the system supports the precise packet manipulation required by adversarial schemes over TCP streams, we implement two critical engineering optimizations:

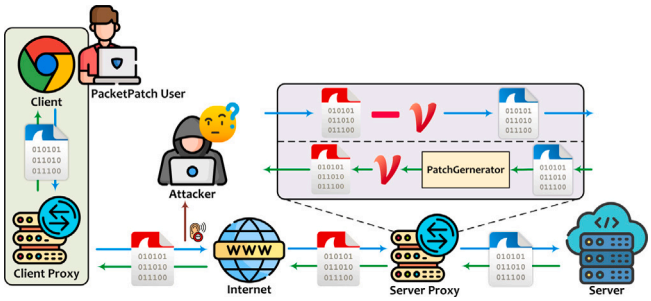


Fig. 6. System deployment based on symmetric proxy architecture.

- **Disabling Nagle Algorithm:** The proxy explicitly disables the Nagle algorithm by setting the `TCP_NODELAY` option. This prevents the operating system from buffering and merging small packets, ensuring that the boundaries of our constructed adversarial packets are preserved during transmission.
- **Packet Delimiters:** Since TCP is a stream-oriented protocol, we inject specific 4-byte delimiters, specifically the sequence `0xff, 0xfe, 0xfd, 0xfc`, into the payload. This allows the receiver to accurately identify the boundaries of adversarial packets within the continuous data stream for correct restoration.

5.2. Adversarial packet construction

The construction of adversarial packets is a pipelined process executed at the sending proxy. To ensure robust evasion and valid transmission, the system strictly follows a three-step procedure: Context Acquisition, Perturbation Generation, and Adversarial Encapsulation.

Step 1: Context Acquisition via Packet Sniffing. The PatchGenerator requires the transport-layer header (TCP/IP header) of the target flow as context to generate effective perturbations. Since the SOCKS5 proxy operates at the application layer and lacks direct access to underlying headers, the system employs a parallel packet sniffer thread utilizing raw sockets. To address the synchronization challenge posed by system latency — where capturing the current packet's header prior to transmission is infeasible — the system retrieves the header of the preceding packet or the TCP handshake packet belonging to the

same 5-tuple flow. By leveraging the temporal stability of flow features, this historical header serves as the effective context input $x_{context}$.

Step 2: Perturbation Generation. The proxy combines the acquired context $x_{context}$ with the current user payload to construct the model input. This input is fed into the TensorRT-accelerated PatchGenerator engine. Upon receiving the input, the engine executes a single forward pass with FP16 precision to generate the perturbation vector v. This vector is structurally composed of three distinct segments: the first padding sequence (Pad_1), a synthesized random header (h_r), and the second padding sequence (Pad_2).

Step 3: Encapsulation and Transmission. To inject the perturbation while maintaining stream integrity, the proxy manipulates the payload buffer before writing to the socket. First, a specific 4-byte delimiter sequence (`0xff, 0xfe, 0xfd, 0xfc`) is prepended to the buffer to enable the receiver to identify the start of an adversarial block within the continuous TCP stream. Subsequently, the generated perturbation vector v is inserted immediately following the delimiter, placing it at the head of the original payload. Finally, the modified buffer is written to the standard socket interface, where the operating system's network stack handles the final encapsulation, automatically calculating checksums and assigning sequence numbers.

5.3. Original packet restoration

The restoration process occurs at the receiving proxy. Its primary objective is to transparently identify adversarial packets within the incoming traffic stream and recover the original payload for the target application.

Step 1: Boundary Identification and Alignment. Upon receiving data from the TCP stream, the receiver scans the buffer for the predefined 4-byte delimiter (`0xff, 0xfe, 0xfd, 0xfc`). The detection of this delimiter signals the beginning of an adversarial packet. This mechanism is crucial for handling potential TCP fragmentation or distinct packet merging, ensuring the receiver always processes aligned data blocks.

Step 2: Perturbation Length Derivation. To minimize overhead, the system does not explicitly transmit the length of the perturbation vector. Instead, the receiver calculates it dynamically by employing a synchronized length mapping rule that is configured identically on both the client and server. Based on the total length of the received data block excluding the delimiter, the receiver computes the precise length of the embedded perturbation L_{total_pad}. This deterministic calculation ensures the accurate separation of perturbation and payload without additional signaling.

Step 3: Stripping and Forwarding. With the boundary and length determined, the restoration is executed. The receiver removes the 4-byte delimiter and the subsequent L_{total_pad} bytes corresponding to the perturbation vector from the head of the data block. The remaining bytes constitute the pristine original payload. This clean data is then forwarded to the destination application server via a standard socket connection, making the entire adversarial process transparent to the upper-layer service.

6. Experiment settings

In this section, we describe the experimental settings in detail, including datasets, B-ETC models, baseline schemes, and evaluation metrics.

6.1. Datasets

To comprehensively validate the effectiveness of PacketPatch, we utilized three publicly available datasets representing distinct application scenarios:

- **ISCX-TOR dataset (TOR)** (Lashkari et al., 2017) contains traffic generated from accessing services via the Tor network. The dataset categorizes traffic into seven service types (e.g., Browsing, Chat). From these, we selected 16 representative applications as target classes for encrypted traffic classification.
- **ISCX-VPN dataset (VPN)** (Draper-Gil et al., 2016) encompasses 11 distinct traffic types, including six types encapsulated through VPN tunnels and five types corresponding to non-VPN traffic accessing the same services.
- **USTC dataset (USTC)** (Wang et al., 2017b) focuses on network security and contains 20 traffic classes, evenly divided into 10 types of malware traffic and 10 types of benign encrypted traffic.

The detailed distribution of categories and sample counts is presented in Table 2. Note that the VPN dataset is balanced via undersampling, whereas the TOR and USTC datasets preserve their original imbalance due to the scarcity of samples in certain categories. Each dataset is partitioned into training, validation, and testing sets in an 8:1:1 ratio.

6.2. B-ETC models

To thoroughly assess the robustness of our method against diverse detection mechanisms, we selected five representative B-ETC models as target models:

- **DeepPacket** (Lotfollahi et al., 2020) is a 1D-CNN-based model that takes raw packet content as input. Each packet is padded or truncated to a unified length of 1500 bytes.
- **LeNet** (Zhou et al., 2017) is a classic 2D-CNN-based model adapted for traffic classification. It reshapes the first 784 bytes of a packet into a 28×28 grayscale image for processing.
- **DataNet** (Wang et al., 2018) is a lightweight MLP-based model. While the original implementation uses 1480 bytes, we normalize the input length to 1500 bytes by padding or truncation to maintain consistency with other baselines.
- **TSCRNN** (Lin et al., 2021) is a hybrid model that combines CNN and RNN. It processes a sequence of 15 consecutive packets, where CNN extracts intra-packet features and Bi-LSTM captures inter-packet temporal dependencies.
- **ET-BERT** (Lin et al., 2022) is a state-of-the-art model based on the BERT architecture. Pre-trained on large-scale unlabeled traffic data, it provides strong feature extraction capability and robustness. It uses the first 512 bytes of each packet as input.

6.3. Baselines

We compare PacketPatch with three baseline methods that generate adversarial perturbations while preserving packet validity. All baselines are white-box methods requiring model gradient or internal state access:

- **ANT** (Sadeghzadeh et al., 2021) computes class-specific Universal Adversarial Perturbations (UAP). We implement two variants, where perturbations are either appended to the end of packets (ANT-e) or injected between the header and payload (ANT-h). The perturbation budget is set to 10% of the original packet length.
- **A-IOT** (Kotak and Elovici, 2023) treats the first 784 bytes of a packet as a grayscale image and employs Class Activation Mapping (CAM) to identify critical bytes. It constructs adversarial packets by retaining the top 10% most important bytes and filling the remaining positions with the target payload.
- **UTAA** (Ding et al., 2024) learns a universal perturbation vector based on internal model activation values. The perturbation is added to the first 784 bytes of each packet, while preserving the packet header to ensure transmission validity.

Table 2
Categories and sample counts for three encrypted traffic datasets.

Dataset	Traffic category (Sample count)
ISCX-TOR (16 Categories)	Facebook(20066), Imap(21000), Vuze(21000), Skype(21183), SFTP(21000), ICQ(710), Spotify(21000), Hangout(20561), Aim(620), POP(21000), Multiplespeed(21000), FTP(21000), SSL(21000), Youtube(21000), Vimeo(21000), Mam(21000)
ISCX-VPN (11 Categories)	Chat(3900), Email(3920), FileTransfer(3363), Streaming(3910), Voip(3036), VPN_Chat(3913), VPN_FileTransfer(3906), VPN_Email(3921), VPN_Streaming(3850), VPN_Torrent(3913), VPN_Voip(3920)
USTC (20 Categories)	BitTorrent(5500), Facetime(3000), FTP(21000), Gmail(7490), MySQL(21000), Outlook(5487), Skype(4014), SMB(21000), Weibo(21000), WorldOfWarcraft(21000), Cridex(20999), Geodo(20811), Htbot(20999), Miuref(20999), Neris(20225), Nsis-ay(21000), Shifu(20999), Tinba(185), Virut(21451), Zeus(18642)

Among the above baselines, ANT and PacketPatch both belong to the perturbation insertion category, where additional bytes are injected into the original packet without modifying existing content. The perturbation budget of ANT is therefore set to 10% of the original packet length, consistent with PacketPatch. In contrast, A-IOT and UTAA operate on fixed-length byte regions, and their fixed-length strategies are integral to their respective method designs. Imposing a percentage-based perturbation budget on these methods would alter their operational mechanisms. All baselines are implemented following their original configurations for faithful reproduction.

6.4. Evaluation metrics

We report standard classification metrics: Accuracy (AC), Precision (PR), Recall (RC), and F1 Score (F1). A decrease in these metrics after perturbation indicates effective privacy protection. Additionally, we use the following specific metrics.

Defense Success Rate (DSR): This metric measures the proportion of packets successfully misclassified by the ETC model after perturbation. Let $\mathbb{D}$ denote the dataset, and let $x \in \mathbb{D}$ represent an original packet with x_{adv} as its adversarial counterpart. With $C(\cdot)$ denoting the classifier's prediction, DSR is calculated as shown in Eq. (8):

$$DSR = \frac{1}{|\mathbb{D}|}|\{x \mid C(x_{adv}) \neq C(x), x \in \mathbb{D}\}| \tag{8}$$

Bandwidth Overhead (BWO): This metric quantifies the additional data volume introduced by the perturbation. Using $Len(\cdot)$ to denote the byte length of a packet, the overhead relative to the original packet x and the adversarial packet x_{adv} is formulated in Eq. (9):

$$BWO = \frac{Len(x_{adv}) - Len(x)}{Len(x)} \tag{9}$$

7. Experiments and analysis

In this section, we conduct a comprehensive experimental evaluation to validate the effectiveness and practicality of PacketPatch. The analysis is structured into three core dimensions: performance evaluation, overhead evaluation, and usability verification. Specifically, we address the following research questions:

- **RQ1 (Performance):** How effective is PacketPatch in evading various ETC models across diverse data scenarios, and how does it compare with state-of-the-art baseline methods? (Section 7.1)
- **RQ2 (Key Factors):** How does the perturbation length influence the attack performance, and what are the distinct contributions of the perturbation vector's individual components to the overall defense success? (Section 7.1)
- **RQ3 (Overhead):** What are the costs associated with PacketPatch in terms of computational latency and bandwidth consumption? (Section 7.2)
- **RQ4 (Usability):** Can the adversarial packets generated by the prototype system be correctly parsed and restored in real-world deployments? (Section 7.3)

7.1. Performance evaluation

This subsection addresses RQ1 and RQ2 and is organized into four parts. First, we evaluate the defense effectiveness of PacketPatch under the default configuration. Second, we compare PacketPatch with representative baseline methods. Third, we conduct a sensitivity analysis to examine how the perturbation length affects the defense performance. Fourth, we perform an ablation study to quantify the contributions of individual components.

7.1.1. Defense effectiveness analysis

We evaluate the performance of five ETC models under PacketPatch, with the detailed results reported in Table 3. The five ETC models perform consistently well across all three datasets. Most methods achieve identification accuracy exceeding 95% in the majority of scenarios, and maintain performance above 85% even under challenging conditions. However, when perturbed by PacketPatch, their performance declines to varying degrees.

The effectiveness of perturbations with identical intensity varies significantly across datasets. Notably, the TOR dataset shows a less significant decline, while the VPN and USTC datasets decline more pronouncedly. This is primarily due to differences in traffic patterns, which influence how easily a perturbation can shift inputs across the decision boundary of the identification model. Besides, the magnitude of decline varies among the different identification models. LeNet exhibits higher degrees of decline, while the ET-BERT displays a certain level of resilience to the perturbations. This can be attributed to the nature of the models. LeNet, originally designed for image classification, struggles with encrypted traffic since converting packet data into image form disrupts the inherent traffic features. In contrast, ET-BERT benefits from extensive pre-training on diverse traffic data, enabling it to form more accurate decision boundaries and generalize better across scenarios.

The notably low DSR against ET-BERT warrants further analysis. Since PatchGenerator is derived from the ET-BERT architecture and leverages its pre-trained weights, the generated perturbations are inherently constrained within the feature manifold learned by ET-BERT. Tokens that PatchGenerator deems least probable via the $\arg\min$ operation and most probable via the $\arg\max$ operation are assessed through the same representational framework that ET-BERT uses for classification. This homology between the generator and the target model makes it inherently difficult for these perturbations to cross the decision boundaries of ET-BERT itself. This constitutes a known limitation of inference-based adversarial methods when the generator and the target model share the same backbone architecture. However, this constraint is of minimal concern in practical deployment, as a real-world defender typically remains agnostic to the specific classification model utilized by an adversary.

7.1.2. Defense effectiveness comparison

We compare PacketPatch with three white-box baseline methods, where the ANT method contains two versions. White-box methods are selected as baselines because they exploit full access to the target model's gradients and internal structures to construct perturbations, representing the theoretical upper bound of adversarial perturbation

Table 3
Effectiveness of PacketPatch.

Dataset	ETC Models	Without PacketPatch				With PacketPatch				
		Acc (%)	Pre (%)	Recall (%)	F1 (%)	Acc (%)	Pre (%)	Recall (%)	F1 (%)	DSR (%)
ISCX-TOR	DeepPacket	99.71	99.71	99.71	99.71	74.68(25.04↓)	74.68(25.03↓)	82.06(17.65↓)	76.00(23.71↓)	25.33
	LeNet	98.92	98.93	98.92	98.92	56.99(41.93↓)	60.35(38.58↓)	56.99(41.93↓)	57.02(41.90↓)	42.99
	DataNet	98.23	98.27	98.23	98.21	73.50(24.73↓)	76.55(21.72↓)	73.50(24.73↓)	73.07(25.15↓)	26.59
	TSCRNN	99.54	99.54	99.54	99.54	74.22(25.32↓)	82.79(16.76↓)	74.22(25.32↓)	72.81(26.73↓)	25.69
	ET-BERT	99.84	99.86	99.84	99.84	99.19(0.65↓)	99.22(0.64↓)	99.19(0.65↓)	99.19(0.65↓)	0.66
ISCX-VPN	DeepPacket	93.87	94.44	93.87	93.85	41.14(52.73↓)	47.19(47.25↓)	41.14(52.73↓)	37.04(56.81↓)	57.37
	LeNet	86.01	86.24	86.01	85.80	14.97(71.04↓)	15.90(70.34↓)	14.97(71.04↓)	8.03(77.78↓)	84.81
	DataNet	85.91	86.27	85.91	85.81	27.29(58.62↓)	31.65(54.62↓)	27.29(58.62↓)	23.78(62.03↓)	70.78
	TSCRNN	98.66	98.63	98.66	98.64	59.02(39.64↓)	56.61(42.02↓)	59.02(39.64↓)	53.38(45.25↓)	40.76
	ET-BERT	98.00	98.02	98.00	98.00	80.53(17.47↓)	82.55(15.47↓)	80.53(17.47↓)	80.61(17.40↓)	18.99
USTC	DeepPacket	98.51	98.53	98.51	98.49	30.30(68.20↓)	46.45(52.08↓)	30.30(68.20↓)	24.25(74.24↓)	69.59
	LeNet	92.93	93.11	92.93	92.79	13.59(79.35↓)	12.70(80.41↓)	13.59(79.35↓)	9.04(83.75↓)	86.22
	DataNet	96.36	96.37	96.36	96.32	25.28(71.07↓)	29.13(67.24↓)	25.28(71.07↓)	16.98(79.34↓)	73.99
	TSCRNN	97.73	97.83	97.73	97.74	4.85(92.89↓)	4.21(93.63↓)	4.85(92.89↓)	2.23(95.51↓)	95.25
	ET-BERT	98.72	98.68	98.72	98.68	51.25(47.46↓)	68.86(29.82↓)	51.25(47.46↓)	46.52(52.17↓)	48.15

Table 4
Comparison between PacketPatch and other baseline methods.

Dataset	ETC Models	ANT-e		ANT-h		A-IOT		UTAA		PacketPatch	
		Acc (%)	DSR (%)	Acc (%)	DSR (%)	Acc (%)	DSR (%)	Acc (%)	DSR (%)	Acc (%)	DSR (%)
ISCX-TOR	DeepPacket	97.59	2.16	**68.32**	**31.57**	72.05	28.00	79.25	20.83	74.68	25.33
	LeNet	97.84	1.39	84.47	15.46	74.05	25.67	81.72	18.39	**56.99**	**42.99**
	DataNet	84.41	14.48	72.18	27.01	66.23	33.71	**56.09**	**43.80**	73.50	26.59
	TSCRNN	82.13	17.72	66.06	33.89	91.17	8.76	**52.74**	**47.09**	74.22	25.69
	ET-BERT	99.84	0.00	99.84	0.00	99.34	0.51	99.84	0.00	**99.19**	**0.66**
ISCX-VPN	DeepPacket	58.11	38.63	36.84	61.63	**33.54**	**66.11**	59.38	40.75	41.14	57.37
	LeNet	78.37	16.04	37.82	60.13	45.68	53.98	14.97	84.81	**14.60**	**85.88**
	DataNet	59.33	33.65	39.45	58.40	36.16	61.32	45.56	56.08	**27.29**	**70.78**
	TSCRNN	77.35	22.23	70.57	29.16	73.63	26.19	63.09	36.86	**59.02**	**40.76**
	ET-BERT	97.28	0.77	72.73	27.00	88.09	11.72	**65.39**	**34.34**	80.53	18.99
USTC	DeepPacket	74.37	24.67	30.50	69.32	47.69	52.21	**27.93**	**71.62**	30.30	69.59
	LeNet	78.91	16.62	30.40	67.95	43.84	53.78	**12.12**	**86.70**	13.59	86.22
	DataNet	75.89	22.04	34.93	64.82	57.36	42.13	**18.40**	**81.47**	25.28	73.99
	TSCRNN	68.91	30.51	75.72	23.34	38.17	61.56	8.83	91.22	**4.85**	**95.25**
	ET-BERT	73.68	25.75	62.54	37.04	75.68	23.71	**33.09**	**66.77**	51.25	48.15

effectiveness. Evaluating PacketPatch against this benchmark provides a rigorous validation of its perturbation generation capability under black-box conditions. The performance of each perturbation method on different datasets and identification models is shown in Table 4. It can be seen that the TOR dataset remains the most challenging to perturb, while ET-BERT models also demonstrate resistance to various white-box methods. Substantially, the effect of perturbations remains uncertain across different scenarios. For instance, the TSCRNN demonstrates robustness in some cases. In the ISCX-VPN dataset, its DSR remains low against all perturbation methods. In the USTC dataset, TSCRNN even outperforms ET-BERT when handling the ANT-h perturbation method. However, in other scenarios, such as under the UTAA model in the ISCX-TOR and USTC scenarios, TSCRNN appears more vulnerable to perturbations. These variations highlight the inherent difficulty of perturbing the byte feature of encrypted traffic. Different models exhibit substantially different decision boundaries when applied to other datasets, which leads to fluctuations in the performance of adversarial packet construction methods across scenarios.

UTAA has the optimal perturbation effect in each scenario because its perturbation surface is the widest, with perturbation vectors added to the first 784 bytes, except for the packet header. However, all other methods choose to insert perturbation vectors at specific positions in the packet, and the original bytes of the packet are not changed, which allows the identification model to still obtain relevant features from the original bytes of the packet.

For a total of 15 scenarios with 5 identification models across 3 datasets, PacketPatch ranks second in terms of the number of times it achieves optimal perturbation effects in each scenario. It leads ANT-e in 15 scenarios, ANT-h in 10 scenarios, A-IOT in 12 scenarios, and UTAA in 8 scenarios, outperforming these white-box methods. These findings demonstrate that PacketPatch, without access to any target model information, can match or exceed the theoretical upper bounds represented by white-box methods in the majority of scenarios.

7.1.3. Perturbation length analysis

Section 4.5 details the rationale behind the length selection of perturbation vectors under the default configuration of PacketPatch from the perspective of method design. This subsection further verifies the rationality of this selection from an experimental perspective. Specifically, we assume that the attacker employs DeepPacket as the ETC model. The random packet header h_r is retained in the perturbation vector, and the total length of Pad_1 and Pad_2 is varied from 0% to 12% of the original packet size. This setup allows us to observe how the length of the perturbation affects attack performance across different datasets.

As shown in Fig. 7, it can be seen that as the length of the perturbation vector increases, the identification accuracy gradually decreases in each data scenario, and the perturbation effect gradually increases. When the filling length increases from 8% to 12%, the growth of perturbation effects in various scenarios tends to flatten out. This is because the total length of Pad_1 and Pad_2 is close to the maximum number of [mask] symbols during training, approaching the limit of PacketPatch generating perturbation vectors. If a longer perturbation vector is needed, the input length of PacketPatch should be expanded to add more reference information to ensure the effectiveness of the perturbation vector. This result validates the rationality of the perturbation vector length selection under the default configuration of PacketPatch.

Table 5
The ablation experiment of PacketPatch.

Method	Component			TOR		VPN		USTC	
	Pad_1	h_r	Pad_2	Acc (%)	DSR (%)	Acc (%)	DSR (%)	Acc (%)	DSR (%)
PacketPatch	✓	✓	✓	74.68	25.33	41.14	57.37	30.30	69.59
$\mathcal{W}^1$	✓	✓	✓	74.38	25.63	42.77	55.25	30.14	69.76
$\mathcal{W}^2$	✓	✓	×	91.21	8.82	**40.28**	**57.93**	**29.97**	**69.90**
$\mathcal{W}^3$	×	✓	✓	**69.43**	**30.53**	42.82	54.86	36.41	63.36
$\mathcal{W}^4$	✓	×	✓	90.92	9.07	50.78	46.07	49.14	50.63
$\mathcal{W}^5$	✓	×	×	91.21	8.82	40.65	57.59	**29.97**	**69.90**
$\mathcal{W}^6$	×	×	✓	74.75	25.20	51.42	45.36	57.91	41.74
$\mathcal{W}^7$	×	✓	×	96.68	3.36	46.44	51.98	37.08	62.67

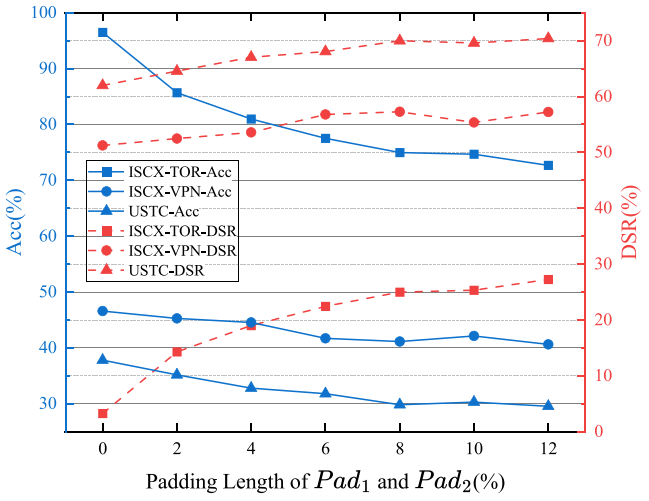


Fig. 7. The impact of PacketPatch with varying perturbation lengths.

7.1.4. Ablation study

The perturbation vector generated by PacketPatch comprises three components: the model-generated segments Pad_1 and Pad_2, separated by an intervening random packet header h_r. The combined length of Pad_1 and Pad_2 is set to 10% of the original packet length, and h_r is a fixed 40-byte real packet header from a random traffic category. To enhance perturbation diversity, the insertion position of this random header is randomized for each instance.

To systematically analyze the contribution of each component, we select DeepPacket as the target ETC model for the ablation study. DeepPacket adopts a 1D-CNN architecture, which is fundamentally different from the BERT-based PatchGenerator and shares no common representations or pre-trained weights. This setting enables a more objective examination of the perturbation mechanism itself, rather than effects that may arise from architectural alignment.

We design seven ablation variants to analyze the contributions of each component from three perspectives: $\mathcal{W}^1$ examines the length allocation strategy between Pad_1 and Pad_2; $\mathcal{W}^2$ through $\mathcal{W}^6$ selectively remove individual components to assess their independent contributions and interactions; and $\mathcal{W}^7$ examines the perturbation capability of h_r in isolation. The detailed configuration of each variant is as follows:

- $\mathcal{W}^1$: All three components are retained, and the total perturbation vector length is equal to the default configuration. Unlike the default configuration where Pad_1 and Pad_2 are randomly assigned their respective lengths, h_r divides Pad_1 and Pad_2 equally to ensure their lengths are identical.
- $\mathcal{W}^2$: Pad_2 is removed by setting its length to zero. Pad_1 and h_r are retained, and the total perturbation vector length remains equal to the default configuration.
- $\mathcal{W}^3$: Pad_1 is removed by setting its length to zero. h_r and Pad_2 are retained, and the total perturbation vector length remains equal to the default configuration.
- $\mathcal{W}^4$: h_r is removed, and the perturbation vector consists of Pad_1 and Pad_2 only. The perturbation vector length is 10% of the original packet length.
- $\mathcal{W}^5$: Only Pad_1 is retained, with the perturbation vector length set to 10% of the original packet length.
- $\mathcal{W}^6$: Only Pad_2 is retained, with the perturbation vector length set to 10% of the original packet length, the same as $\mathcal{W}^4$ and $\mathcal{W}^5$.
- $\mathcal{W}^7$: Only the random header h_r is used as the perturbation vector, with a fixed length of 40 bytes.

The results are shown in Table 5. Regarding the individual contributions of Pad_1 and Pad_2, comparisons between $\mathcal{W}^2$ and $\mathcal{W}^3$ alongside comparisons between $\mathcal{W}^5$ and $\mathcal{W}^6$ reveal a clear divergence: Pad_2 contributes more significantly to perturbation effectiveness in the ISCX-TOR dataset, whereas Pad_1 demonstrates greater impact in the ISCX-VPN and USTC datasets. This discrepancy is attributable to the structure of the perturbation vector: Pad_1 is separated from h_r by a [SEP] token, which causes the PatchGenerator to treat them as independent, resulting in a lack of semantic correlation between Pad_1 and the header. Conversely, Pad_2 is generated based on both the random header and the original payload, establishing a strong feature dependency. Consequently, the relative dominance of these two components varies across data scenarios, reflecting the scenario-dependent nature of the perturbation effect.

We further examine the role of h_r. Observations from $\mathcal{W}^7$ indicate that randomly selected packet headers alone can perturb packet byte features to some extent, although their effectiveness is limited in the ISCX-TOR dataset. Comparisons between $\mathcal{W}^1$ and $\mathcal{W}^4$ confirm that h_r contributes meaningfully to the overall perturbation vector. Further analysis comparing $\mathcal{W}^5$ with $\mathcal{W}^2$ reveals that adding h_r to Pad_1 leads to minimal improvement, suggesting weak feature correlation. In contrast, the comparison between $\mathcal{W}^6$ and $\mathcal{W}^3$ demonstrates that adding h_r to Pad_2 results in a substantial boost in perturbation effectiveness. This finding further confirms the structural linkage between Pad_2 and h_r, which significantly enhances the overall perturbation effect.

It is also important to address why the default configuration of PacketPatch does not achieve the highest DSR in any individual scenario. As analyzed above, the optimal component varies by scenario: Pad_2 dominates in ISCX-TOR, while Pad_1 is more effective in ISCX-VPN and USTC. Consequently, specialized variants such as $\mathcal{W}^3$, which excludes Pad_1, and $\mathcal{W}^2$, which excludes Pad_2, achieve the highest DSRs in their respective favorable scenarios, reaching 30.53% on the TOR dataset and 57.93% and 69.90% on the VPN and USTC datasets, respectively. Since the default configuration distributes the perturbation budget across both Pad_1 and Pad_2 via random allocation, it inevitably allocates some capacity to the less effective component in each specific scenario, resulting in slightly suboptimal performance compared to the scenario-specific best variant. Nevertheless, the default configuration consistently maintains competitive performance across all three datasets with DSR values of 25.33%, 57.37%, and 69.59%, closely approaching the scenario-specific optima. Moreover, compared with $\mathcal{W}^1$ which employs equal division, the default random allocation achieves notably better performance on the ISCX-VPN dataset, surpassing the 55.25% accuracy of $\mathcal{W}^1$ with 57.37%. This suggests that

Table 6
The time consumption of PacketPatch.

Perturbation stage	Time consumption (ms/packet)
Packet Preprocessing	0.054
Perturbation Generation	9.626
Packet Reconstruction	0.398
Total	10.078

stochastic length variation introduces beneficial perturbation diversity. These observations demonstrate that the default configuration offers a robust and balanced trade-off between generality and effectiveness. When the deployment environment is uncertain, this configuration serves as a reliable general-purpose strategy. However, if the maximum perturbation effect is desired for a specific target, the components can be adaptively configured to suit the traffic characteristics and identification models.

Although the ablation study is conducted on a single target model, the observed trends are mainly determined by the construction of the perturbation vector and the generation process of PatchGenerator, rather than being specific to DeepPacket. Specifically, Pad_1 is generated under the original header context and tends to introduce feature deviations from the original packet representation, while Pad_2 is generated conditioned on both the random header and the original payload, leading to stronger feature dependency. As these behaviors arise from how the perturbation is constructed and generated, similar relative trends are expected when applied to other target models, although the overall perturbation strength may vary depending on model sensitivity.

7.2. Overhead evaluation

This subsection answers RQ3. We first evaluate the time efficiency of PacketPatch by reporting its time consumption. Subsequently, we quantify the bandwidth overhead and compare these costs against representative baseline methods.

7.2.1. Time efficiency analysis

We separately calculate the time consumption for each stage of the PacketPatch application process, which is divided into three phases: (1) packet preprocessing, where the system processes the packet into the model input format; (2) perturbation vector generation, where the PatchGenerator performs a single forward pass (with the batch size set to 1 to measure per-sample latency); and (3) adversarial packet construction, where the perturbation vector is inserted into the target packet and the relevant header fields are updated.

As shown in Table 6, the time consumption of PacketPatch mainly occurs during the perturbation vector generation stage, which necessitates a model forward pass. Overall, constructing an adversarial packet using PacketPatch incurs an average latency of approximately 10 ms, demonstrating its feasibility for real-time applications.

7.2.2. Bandwidth overhead analysis

We compare the actual bandwidth consumption of various methods across different datasets, as illustrated in Fig. 8. The two versions of ANT exhibit identical BWO. In contrast, A-IOT demonstrates significantly higher BWO than other methods, as it mandates padding packets to a fixed template length regardless of the original packet size. Similarly, UTAA requires padding smaller packets to a fixed size of 784 bytes. PacketPatch incurs a slightly higher BWO than ANT, which is attributable to the inclusion of an additional packet header in its perturbation vector.

Overall, PacketPatch achieves comparable or superior perturbation effects relative to the three white-box baselines, even under black-box assumptions. Furthermore, it effectively balances perturbation effectiveness with bandwidth overhead.

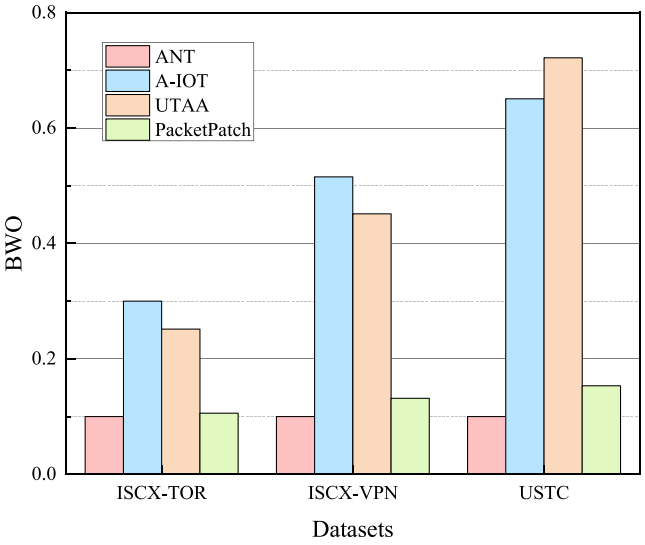


Fig. 8. BWO of various methods under different datasets.

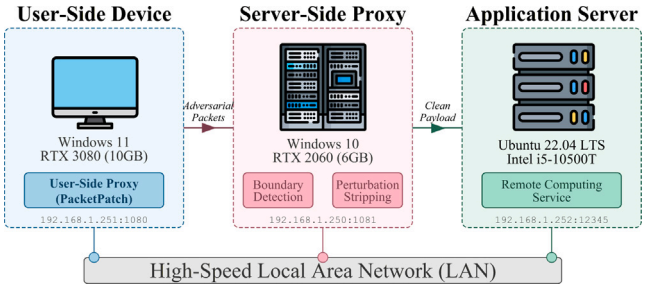


Fig. 9. The architecture and deployment of the PacketPatch prototype system.

7.3. Usability verification

This subsection answers RQ4. Prototype system deployment presents the end-to-end prototype system and evaluation setting, and Packet Usability Verification demonstrates parsability and restorability with a packet-level example.

7.3.1. Prototype system deployment

To strictly verify the practical usability and restorability of adversarial packets in a real-world network environment, we instantiate the symmetric proxy architecture described in Section 5. As illustrated in Fig. 9, the experimental testbed comprises three distinct physical nodes: the user-side device, the server-side proxy, and the application server, all interconnected within a high-speed local area network (LAN).

The specific hardware and software configurations for each node are detailed in Table 7. The user-side device is equipped with a high-performance NVIDIA GeForce RTX 3080 GPU to support the real-time inference requirements of the PacketPatch module. The server-side proxy is deployed on a separate workstation with an RTX 2060 GPU to handle the removal of perturbations. The application server hosts a custom remote computing service on an Ubuntu environment to process user requests.

The network topology is configured to simulate a complete end-to-end communication link, as illustrated by the following IP and port configurations:

- The user-side device (IP: `192.168.1.251`) runs the client proxy, which listens on the local loopback port `1080`. It intercepts outbound connection requests and encapsulates them with adversarial perturbations.

Table 7
Hardware and software configuration of the prototype system.

Device	Component	Specification
User-Side Device	OS	Windows 11 Home
	CPU	Intel(R) Core(TM) i9-12900 CPU @ 3.19 GHz
	Memory	64.0 GB
	GPU (VRAM)	NVIDIA GeForce RTX 3080 (10 GB)
	Network	Intel Wi-Fi 6 AX201 160MHz
Server-Side Proxy	OS	Windows 10 Pro
	CPU	Intel(R) Core(TM) i9-10900 CPU @ 2.80 GHz
	Memory	32.0 GB
	GPU (VRAM)	NVIDIA GeForce RTX 2060 (6 GB)
	Network	Intel(R) Wi-Fi 6 AX200 160MHz
Application Server	OS	Ubuntu 22.04 LTS
	CPU	Intel Core i5-10500T @ 2.3 GHz
	Memory	8.0 GB
	Network	Comfast CF-952AX WIFI6 USB

- The traffic is forwarded to the server-side proxy (IP: `192.168.1.250`), which listens on port `1081`. This node is responsible for boundary detection and perturbation stripping.
- Upon restoration, the clean payload is forwarded to the application server (IP: `192.168.1.252`), where the remote computing service listens on port `12345`.

To definitively answer RQ4, we design a rigorous verification protocol using the deployed remote computing service. The client application sends 1000 continuous mathematical calculation problems to the application server, with random transmission intervals ranging from 0.5 ms to 5 ms.

In this experiment, the successful receipt of correct calculation answers by the client serves as the ground truth for validating usability. This metric implies two critical outcomes:

- *Parsability*: The adversarial packets successfully traverse the network stack and are correctly parsed by the server-side proxy, proving that the injected headers and delimiters comply with protocol standards.
- *Restorability*: The server-side proxy successfully identifies the perturbation boundaries and restores the original payload to its bit-perfect state. Any error in restoration results in an incorrect calculation answer or a service timeout.

The experimental results confirm a 100% success rate for the 1000 calculation tasks, thereby validating that the adversarial packets generated by the prototype system satisfy the strict correctness requirements of real-world deployments.

7.3.2. Packet usability verification

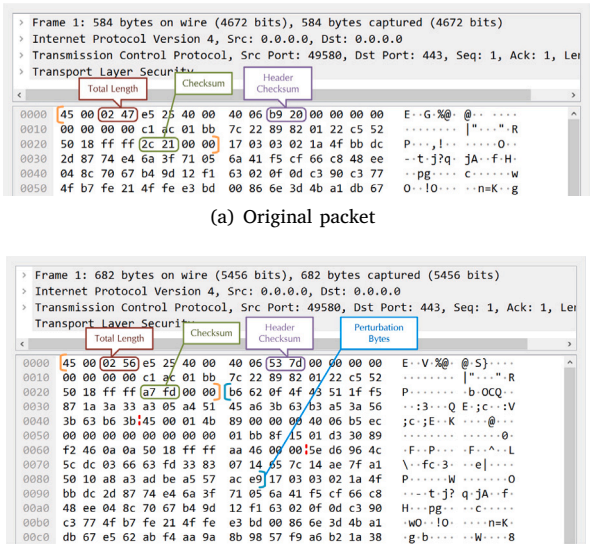
Fig. 10 illustrates the generation of an adversarial packet using PacketPatch. As shown in Fig. 10(a), the original packet size is 584 bytes. Fig. 10(b) displays the generated perturbation vector of 98 bytes, which comprises 58 bytes generated by PacketPatch and a 40-byte h_r (separated by red dashed lines).

The perturbation vectors are inserted between the header and the payload of the original packet. After inserting the perturbation vectors, critical fields in the original packet are recalculated, including the packet length in the IP header, the IP header checksum, and the checksum of the entire packet in the TCP header. This ensures that the receiver can calculate the original length of the packet upon receiving it and remove the added perturbation vectors, facilitating correct parsing and restoration.

8. Conclusion

This study focuses on the generation and deployment of adversarial packets for B-ETC to effectively avert privacy violations linked to user network traffic. In response to the deficiencies identified in existing research, we present PacketPatch, a practical approach for generating and deploying adversarial packets aimed at B-ETC Models. To avoid setting unrealistic preconditions, we design PatchGenerator, a BERT-based perturbation vector generator that operates under a black-box assumption. It produces perturbations from original packets and random header data without requiring access to model gradients or user behavior history. To reduce time and bandwidth overhead, PatchGenerator produces perturbations with a single forward pass and dynamically varies the number of masked bytes based on the packet size, effectively managing the length of the perturbation. To ensure packet usability, we further propose a symmetric proxy-based deployment method. By strategically placing the perturbation vector at the boundary between the packet header and payload, while also updating the relevant protocol fields, adversarial packets can successfully pass integrity checks and be seamlessly restored at the receiving end.

To validate PacketPatch, we rigorously tested its effectiveness in defense against five B-ETC models across three datasets. It outperforms four leading white-box baselines in most scenarios while maintaining a strict black-box approach. Its average perturbation vector generation time is just 10.078 ms, with much lower bandwidth overhead compared to template-filling methods. Furthermore, we built a prototype system and deployed PacketPatch on it. The test results indicate that PacketPatch could bypass B-ETC models and successfully recover the original information at the receiving end. Our future studies will investigate innovative approaches to the generation of general adversarial packets and the frameworks for their deployment. By generating a common perturbation vector for packets belonging to the same category, we hope to cut down the generator's resource use while keeping our defense effectiveness.



(a) Original packet

(b) Packet perturbed by PacketPatch

Fig. 10. An example of a packet perturbed by PacketPatch.

CRediT authorship contribution statement

Yuwei Xu: Writing – review & editing, Supervision, Resources, Project administration, Conceptualization. **Yuanyuan Xu:** Writing – review & editing, Writing – original draft, Visualization, Validation, Investigation, Formal analysis. **Yunpeng Bai:** Writing – original draft, Software, Methodology, Formal analysis, Data curation. **Jiahui Chen:** Writing – review & editing, Validation. **Kehui Song:** Writing – review & editing, Visualization. **Jie Cao:** Writing – review & editing, Validation. **Qiao Xiang:** Writing – review & editing, Validation. **Guang Cheng:** Writing – review & editing, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the CCF-NSFOCUS 'Kunpeng' Research Fund, China under Grant No. CCF-NSFOCUS 2024010, in part by the National Key R&D Program of China under Grant No. 2023YFB3106700, and in part by the National Natural Science Foundation of China under Grant Nos. 62172093 and U22B2025.

Data availability

Data will be made available on request.

References

Aceto, G., Ciuonzo, D., Montieri, A., Pescapé, A., 2019. Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges. IEEE Trans. Netw. Serv. Manag. 16 (2), 445–458.

Aouedi, O., Le, V.A., Piamrat, K., Ji, Y., 2025. Deep learning on network traffic prediction: Recent advances, analysis, and future directions. ACM Comput. Surv. 57 (6).

Cao, J., Xu, Z., Bai, Y., et al., 2024. Perturbing vulnerable bytes in packets to generate adversarial samples resisting DNN-based traffic monitoring. In: Proceedings of the 23rd IEEE International Conference on Trust, Security and Privacy in Computing and Communications, TrustCom. IEEE, pp. 1015–1021.

Cheng, J., Wu, Y., E, Y., et al., 2021. MATEC: A lightweight neural network for online encrypted traffic classification. Comput. Netw. 199, 108472.

Dai, X., Liang, K., Xiao, B., 2024. Advdiff: Generating unrestricted adversarial examples using diffusion models. In: Proceedings of the 18th European Conference on Computer Vision. ECCV, Springer, pp. 93–109.

Devlin, J., Chang, M.-W., Lee, K., Toutanova, K., 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT. pp. 4171–4186.

Ding, R., Sun, L., Zang, W., et al., 2024. Towards universal and transferable adversarial attacks against network traffic classification. Comput. Netw. 254, 110790.

Ding, Y., Zhu, G., Chen, D., et al., 2022. Adversarial sample attack and defense method for encrypted traffic data. IEEE Trans. Intell. Transp. Syst. 23 (10), 18024–18039.

Dong, W., Yu, J., Lin, X., et al., 2025. Deep learning and pre-training technology for encrypted traffic classification: A comprehensive review. Neurocomputing 617, 128444.

Draper-Gil, G., Lashkari, A.H., Mamun, M.S.I., Ghorbani, A.A., 2016. Characterization of encrypted and vpn traffic using time-related. In: Proceedings of the 2nd International Conference on Information Systems Security and Privacy. ICISSP, pp. 407–414.

He, H., Yang, Z., Chen, X., 2020. PERT: Payload encoding representation from transformer for encrypted traffic classification. In: Proceedings of the 12th ITU Kaleidoscope Academic Conference, ITU K. IEEE, pp. 1–8.

Hore, S., Ghadermazi, J., Paudel, D., et al., 2025. Deep packgen: A deep reinforcement learning framework for adversarial network packet generation. ACM Trans. Priv. Secur. 28 (2), 1–33.

Hu, X., Zhu, C., Cheng, G., et al., 2023. A deep subdomain adaptation network with attention mechanism for malware variant traffic identification at an IoT edge gateway. IEEE Internet Things J. 10 (5), 3814–3826.

Kotak, J., Elovici, Y., 2023. Adversarial attacks against IoT identification systems. IEEE Internet Things J. 10 (9), 7868–7883.

Lashkari, A.H., Gil, G.D., Mamun, M.S.I., Ghorbani, A.A., 2017. Characterization of tor traffic using time based features. ICISSP, In: Proceedings of the 3rd International Conference on Information Systems Security and Privacy, vol. 2, pp. 253–262.

Lin, X., Xiong, G., Gou, G., et al., 2022. ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In: Proceedings of the 31st ACM Web Conference. WWW, pp. 633–642.

Lin, K., Xu, X., Gao, H., 2021. TSCRNN: A novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT. Comput. Netw. 190, 107974.

Lotfollahi, M., Jafari Siavoshani, M., Shirali Hossein Zade, R., Saberian, M., 2020. Deep packet: A novel approach for encrypted traffic classification using deep learning. Soft Comput. 24 (3), 1999–2012.

Papadogiannaki, E., Ioannidis, S., 2021. A survey on encrypted network traffic analysis applications, techniques, and countermeasures. ACM Comput. Surv. 54 (6), 1–35.

Sadeghzadeh, A.M., Shiravi, S., Jalili, R., 2021. Adversarial network traffic: Towards evaluating the robustness of deep-learning-based network traffic classification. IEEE Trans. Netw. Serv. Manag. 18 (2), 1962–1976.

Sharma, A., Lashkari, A.H., 2025. A survey on encrypted network traffic: A comprehensive survey of identification/classification techniques, challenges, and future directions. Comput. Netw. 257, 110984.

Shen, M., Ye, K., Liu, X., et al., 2022. Machine learning-powered encrypted network traffic analysis: A comprehensive survey. IEEE Commun. Surv. Tutor. 25 (1), 791–824.

Sun, P., Yun, X., Li, S., et al., 2025. AdvTG: An adversarial traffic generation framework to deceive DL-based malicious traffic detection models. In: Proceedings of the 34th ACM Web Conference. WWW, ACM, pp. 3147–3159.

Wang, P., Ye, F., Chen, X., Qian, Y., 2018. Datanet: Deep learning based encrypted network traffic classification in sdn home gateway. IEEE Access 6, 55380–55391.

Wang, W., Zhu, M., Wang, J., et al., 2017a. End-to-end encrypted traffic classification with one-dimensional convolution neural networks. In: Proc. of the 15th IEEE International Conference on Intelligence and Security Informatics. ISI, IEEE, pp. 43–48.

Wang, W., Zhu, M., Zeng, X., et al., 2017b. Malware traffic classification using convolutional neural network for representation learning. In: Proceedings of the 31st International Conference on Information Networking. ICOIN, IEEE, pp. 712–717.

Wu, H., Liu, Y., Cheng, G., Hu, X., 2023. RT-CBCH: Real-time VPN traffic service identification based on sampled data in high-speed networks. IEEE Trans. Netw. Serv. Manag. 21 (1), 88–107.

Ye, M., Chen, J., Miao, C., et al., 2023. PAT: Geometry-aware hard-label black-box adversarial attacks on text. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. SIGKDD, pp. 3093–3104.

Zhang, X., Zhou, Y., Pei, S., et al., 2020. Adversarial examples detection for XSS attacks based on generative adversarial networks. IEEE Access 8, 10989–10996.

Zhou, Y., Shi, H., Zhao, Y., et al., 2021. Encrypted network traffic identification based on 2d-CNN model. In: Proceedings of the 22nd Asia-Pacific Network Operations and Management Symposium. APNOMS, IEEE, pp. 238–241.

Zhou, H., Wang, Y., Lei, X., Liu, Y., 2017. A method of improved CNN traffic classification. In: Proceedings of the 13th International Conference on Computational Intelligence and Security. CIS, pp. 177–181.

Zhu, S., Xu, X., Gao, H., Xiao, F., 2023. CMTSNN: A deep learning model for multiclassification of abnormal and encrypted traffic of internet of things. IEEE Internet Things J. 10 (13), 11773–11791.